

NeoRent Rental Platform

Comprehensive Technical Implementation & Architectural Report

This document presents the exhaustive end-to-end technical implementation details for the **NeoRent** Next-Generation Equipment & Asset Rental Platform developed for the **Odoo Hackathon 2026 ASPD Project**. It encompasses the dual API architecture, frontend/backend technology implications, SQLite3 relational database schema, end-to-end business workflows, modular component design, complete GitHub commit history, and critical engineering challenges along with their resolutions.

Project Name:	NeoRent Rental Platform (Odoo Hackathon 2026 ASPD)
Repository:	Odoo_Hackathon2026_Final_Round_ASPD_Project
Authors / Team:	avrojitsaha763-byte (Avrojit Saha)
Primary Tech Stack:	React 18, Vite, Tailwind CSS, FastAPI, Django, SQLite3, SQLAlchemy
Document Date:	August 9, 2026
Document Status:	Final Audited & Hardened Release

EXECUTIVE SUMMARY: NeoRent resolves key operational challenges in asset rentals—including dynamic time-unit pricing, automated quotation-to-order lifecycles, variant attribute management, real-time IoT smart lock telemetry, and fraud risk detection—by leveraging a high-performance React 18 SPA coupled with a resilient FastAPI / Django dual backend and SQLite3 storage.

Table of Contents

Section 1	1. Use of API Implications & Specifications	Dual REST Architecture, FastAPI & Third-Party Browser APIs
Section 2	2. Tech Implications of Frontend & Backend	React 18 SPA, Tailwind CSS, Dual FastAPI/Django Engine & Security
Section 3	3. SQLite3 Database Schema Description	Relational Models, FastAPI Tables, Django Core ORM, JSON Fields
Section 4	4. Whole Project Flow from Beginning to Ending	Customer & Admin Workflows, Lifecycles & State Transitions
Section 5	5. Implementation of Modules	Comprehensive Breakdown of Frontend & Backend Components
Section 6	6. GitHub Progress & Commit History	Chronological Repository Trajectory & Development Milestones
Section 7	7. Challenges Faced, Bugs, Issues & Resolutions	Deep Technical Post-Mortem & Engineering Fixes

1. Use of API Implications & Specifications

NeoRent employs a dual REST API architecture. The primary runtime microservice relies on **FastAPI** executing via Uvicorn on `http://localhost:8000`, providing high-concurrency async endpoints. In parallel, a full **Django REST Framework** backend is retained under `backend/core` for Django administrative management, database migrations, and alternative ORM mapping.

1.1 Core FastAPI REST Endpoints

Method & Endpoint	Input Payload / Params	Description & Operations
GET /api/products	Query: category, search, published	Retrieves list of all published/all products with parsed pricing rates, colors, and variant specifications.
POST /api/products	JSON Body: Product Schema	Creates a new rental product in SQLite database. Generates unique ID if omitted.
PUT /api/products/{id}	JSON Body: Product fields	Updates existing product specifications, stock levels, sales prices, and periodicities.
DELETE /api/products/{id}	Path: Product ID	Removes product record from SQLite database.
GET /api/pricelists	None	Fetches active pricelists with rules for quantity discounts and fixed price overrides.
GET /api/attributes	None	Fetches global product attribute definitions (Radio, Pills, Checkbox, Image).
GET /api/templates	None	Retrieves quotation templates with validity days and payment terms percentages.
POST /api/rental-orders	JSON: Order payload	Creates a new rental order (Quotation or Sale Order) with items, customer details, and pricing calculations.
PUT /api/rental-orders/{id}	JSON: Order updates	Updates rental order status ('QUOTATION', 'SALE_ORDER') or logistics state ('TODAY', 'PICKUP', 'RETURN', 'LATE').
GET /api/invoices	None	Fetches customer invoices associated with rental orders.
POST /api/invoices	JSON: Invoice schema	Generates a draft or posted invoice from a completed rental sale order.
GET /api/iot-telemetry	None	Retrieves real-time smart lock GPS coordinates, battery status, lock state, and tamper alerts.
GET /api/audit-logs	None	Retrieves security and operational audit trail logs recorded across system actions.
POST /api/auth/admin-signup	JSON: {email, password}	Verifies admin authorization against allowed administrator domains (neorent435@gmail.com).

1.2 Third-Party & Browser API Implications

- **Web Speech Recognition API:** Integrated in VoiceCommandPanel.jsx for hands-free voice command parsing (e.g., 'Search DSLR Camera', 'Go to Admin Dashboard', 'Switch currency to EUR').
- **Stripe Payment Gateway (Mock Integration):** Embedded within CheckoutView.jsx providing asynchronous payment processing simulation, card element tokenization, and instant confirmation callback.
- **Canvas Confetti Particle Engine:** Integrated via src/utils/celebrate.js to trigger real-time canvas-rendered celebratory confetti explosions upon successfully confirming a rental order.
- **HTML5 Media Streams:** Utilized in VideoHero.jsx for rendering ambient high-definition background video streams with inline fallback image handling.
- **Browser LocalStorage API:** Serves as an offline state cache and persistence mechanism in AppContext.jsx, allowing full application usability even during backend disconnects.

2. Tech Implications of Frontend & Backend

2.1 Frontend Technical Architecture

The frontend is engineered as a Single Page Application (SPA) using **React 18** bundled with **Vite** for sub-millisecond hot module replacement (HMR). Styling is governed by **Tailwind CSS v3** with custom keyframe animations, glassmorphism utilities (backdrop blurs, semi-transparent borders), and custom color tokens.

- **State Management Architecture:** Centralized state is maintained through React Context API (AppContext.jsx). State synchronization operates on an optimistic update strategy—UI updates immediately while background API calls reconcile state asynchronously.
- **Component Architecture:** Modular presentation and container components. Clean separation between storefront view, configuration modals, checkout workflows, and administrative management modules.
- **Dynamic Multi-Currency Engine:** Front-end currency context converts all monetary values dynamically across USD (\$), EUR (€), GBP (£), and INR (₹) using real-time conversion coefficients.

2.2 Backend Technical Architecture

The backend leverages a dual-framework setup to maximize agility and compliance with hackathon mandates:

1. **FastAPI Asynchronous Microservice (backend/main.py):** Lightweight ASGI framework utilizing Pydantic data schemas for request validation, custom CORS middleware, and SQLAlchemy ORM for SQLite database interaction.
2. **Django Web Application (backend/core):** Standard Django project structure featuring model definitions in models.py, serializers in serializers.py, and REST framework viewsets in views.py for Django admin access.

2.3 Security & Role-Based Access Control (RBAC)

NeoRent enforces three strict user roles: **CUSTOMER**, **VENDOR**, and **ADMIN**. Admin privileges (accessing IoT Telemetry, modifying security deposits, overriding pricelists) require authentication against authorized domain credentials (e.g., neorent435@gmail.com). Unverified registration attempts are instantly blocked at the API level with audit log entries recorded.

3. SQLite3 Database Schema Description

NeoRent's data architecture is backed by an **SQLite3** database (db.sqlite3). It combines traditional relational tables with a document-hybrid approach using stringified JSON fields for flexible specs, multi-currency rates, order lines, and IoT payload storage.

Table Name	Primary Key	Columns & Types	Description & Relations
products	id (VARCHAR)	name, brand, colors, price (TEXT JSON), category, image, product_type, stock_quantity (FLOAT), sales_price, cost_price, security_deposit, periodicity, is_published (BOOL), hasVariants, specs (TEXT JSON)	Stores equipment rental catalog. price holds rate tiers; specs holds key-value specs.
attributes	id (VARCHAR)	name (VARCHAR), display_type (VARCHAR: RADIO/PILLS/CHECKBOX/IMAGE), values (TEXT JSON)	Global product variant attributes (e.g., Lens Type, Color, Ram Size).
pricelists	id (VARCHAR)	name (VARCHAR), rules (TEXT JSON: product_id, discount_pct, fixed_price, min_qty)	Dynamic pricing rules for promotional discounts or bulk rental rates.
templates	id (VARCHAR)	name (VARCHAR), validity_days (INT), payment_terms_percentage (FLOAT), lines (TEXT JSON)	Quotation templates for standard rental agreements and payment terms.
rental_orders	orderId (VARCHAR)	customerName, customerEmail, customerPhone, deliveryMethod, status (QUOTATION/SALE_ORDER), totalAmount (FLOAT), rentalPeriodStart, rentalPeriodEnd, deliveryAddress (JSON), billingAddress (JSON), orderLines (JSON), paymentDetails (JSON), invoices (JSON)	Central order repository tracking quotation sent, sale confirmed, and logistics state.
invoices	invoiceNumber	orderId (VARCHAR), issueDate, invoiceStatus (Draft/Posted/Paid), amountDue (FLOAT), invoiceLines (TEXT JSON)	Financial invoices associated 1:1 or 1:N with confirmed rental sale orders.
audit_logs	id (VARCHAR)	timestamp (VARCHAR), action (VARCHAR), user (VARCHAR), details (TEXT)	Immutable security audit log capturing user authentication, order status changes, and catalog edits.
iot_telemetry	deviceId (VARCHAR)	deviceName, lat (FLOAT), lng (FLOAT), battery (FLOAT), status (Active/Alert), lastPing	Real-time telemetry for smart lock IoT GPS trackers embedded in high-value rental items.

3.2 Django Core ORM Schema (backend/core/models.py)

In addition to the FastAPI SQLite tables, the Django backend defines strict normalized relational models: User (extending AbstractUser), VendorProfile, QuotationTemplate, ProductAttribute, ProductAttributeValue, Product, ProductVariant, Pricelist, PricelistRule, Wishlist, Cart, CartItem, RentalOrder, OrderLineItem, and Invoice.

4. Whole Project Flow from Beginning to Ending

4.1 Customer User Journey (End-to-End)

- Step 1: Landing & Hero Discovery:** Customer lands on the platform with video hero background (VideoHero.jsx), interactive search bar, category quick filters, and live multi-currency selection.
- Step 2: Storefront Exploration & Filtering:** Customer filters products by Category (Goods vs Services), Brand, Price Range, and Availability (SidebarFilters.jsx). Risk badges indicate high-value security deposit items.
- Step 3: Product Configuration & Date Selection:** Clicking 'Configure Rental' opens ProductConfigModal.jsx. Customer selects attribute variants (e.g. 4K Camera Body, 24-70mm Lens), rental periodicity (Hourly, Daily, Weekly, Nightly), and rental start/end datetimes.
- Step 4: Real-Time Pricing & Security Deposit Calculation:** The configuration modal updates the live total price = (Base Rate * Period Multiplier) + Variant Modifiers + Security Deposit. Risk Oracle calculates fraud/loss risk score.
- Step 5: Shopping Cart Management:** Items are committed to CartView.jsx. Customer can review rental breakdown, modify quantities, update rental dates, or input promotional discount codes.
- Step 6: Checkout & Payment Verification:** In CheckoutView.jsx, customer enters delivery/billing address, selects shipping method (Standard/Express/Pickup), and completes payment via simulated Stripe card checkout.
- Step 7: Order Confirmation & Confetti Celebration:** Upon success, celebrate.js triggers canvas confetti. OrderConfirmation.jsx renders order summary, rental agreement terms, and printable invoice PDF download link.
- Step 8: Post-Order Tracking & AI Assistance:** Customer tracks order progress on RentalJourneyTimeline.jsx (Quotation -> Sale Order -> Out for Delivery -> Active Rental -> Returned). NeoBotChat.jsx provides 24/7 AI assistance.

4.2 Vendor & Administrator Workflow

- Step 1: Admin Authentication:** Administrator logs in via AuthFlow.jsx. Restricted email domains (neorent435@gmail.com) grant elevated permissions.
- Step 2: Admin Dashboard Management:** Admin enters AdminDashboard.jsx containing 8 dedicated management hubs (Products, Orders, Quotations, Invoices, Pricelists, Attributes, IoT Telemetry, Audit Logs).
- Step 3: Catalog & Attribute Administration:** Admin creates/edits equipment listings, stock levels, periodicity rates, and variant attribute matrices.
- Step 4: Quotation to Order Conversion:** Admin reviews customer quotations, updates quotation validity/payment terms, sends quotations to customers, and confirms sale orders.
- Step 5: Invoice Lifecycle & Logistics Dispatch:** Admin posts draft invoices, tracks payment status, and updates rental logistics status ('TODAY', 'PICKUP', 'RETURN', 'LATE').
- Step 6: IoT Telemetry & Asset Security:** Admin monitors real-time GPS coordinates, battery levels, and tamper alerts of active rental equipment via the IoT Telemetry map.

5. Implementation of Modules

5.1 Frontend Components Breakdown

Module Name	File Location	Key Features & Responsibilities
LandingPage	src/components/LandingPage.jsx	Renders hero banner, category showcases, featured rental products, customer testimonials, and feature highlight grid.
VideoHero	src/components/VideoHero.jsx	HTML5 autoplay background video player with overlay gradient, call-to-action buttons, and mobile fallback image.
Navbar	src/components/Navbar.jsx	Top navigation bar with search bar, multi-currency switcher dropdown, cart drawer trigger, role badge, and auth buttons.
SidebarFilters	src/components/SidebarFilters.jsx	Dynamic catalog filtering panel: search query, category selection, price slider, brand checkboxes, and sort order.
ProductCard	src/components/ProductCard.jsx	Product card with hover animations, stock status badge, rental periodicity rate display, risk indicator badge, and quick rent button.
ProductConfigModal	src/components/ProductConfigModal.jsx	Interactive modal for selecting variant attributes, date range picker, duration calculator, security deposit preview, and add-to-cart.
CartView	src/components/CartView.jsx	Shopping cart drawer/page displaying itemized rental summary, duration multipliers, subtotal calculation, and checkout CTA.
CheckoutView	src/components/CheckoutView.jsx	Multi-step checkout workflow with customer billing/shipping details, logistics selection, Stripe mock payment form, and order submission.
OrderConfirmation	src/components/OrderConfirmation.jsx	Post-checkout receipt page with order summary, rental period dates, payment reference, and confetti celebration trigger.
AdminDashboard	src/components/AdminDashboard.jsx	Comprehensive vendor management console with 8 tabs for catalog, orders, quotations, invoices, pricelists, attributes, IoT, and audit logs.
AuthFlow	src/components/AuthFlow.jsx	Authentication modal supporting Customer login, Vendor sign-in, and restricted Admin sign-up with email verification.
NeoBotChat	src/components/NeoBotChat.jsx	Floating AI chatbot assistant answering rental policies, pricing rules, order status inquiries, and equipment guidance.
VoiceCommandPanel	src/components/VoiceCommandPanel.jsx	Speech recognition panel processing natural voice queries to navigate pages, filter products, and control application settings.
RentalJourneyTimeline	src/components/RentalJourneyTimeline.jsx	Visual progress tracker displaying real-time rental order status milestones from quotation creation to final asset return.
RiskBadge & riskOracle	src/utls/riskOracle.js	Algorithmic engine scoring transaction risk based on item value, security deposit ratio, rental duration, and user history.
CountdownTimer	src/components/CountdownTimer.jsx	Live countdown timer warning customers of reservation hold expirations for high-demand rental items.
AppContext	src/context/AppContext.jsx	Centralized state context providing global state, cart operations, multi-currency conversion, local storage cache, and backend API sync.

5.2 Backend Modules Breakdown

- **FastAPI Microservice (backend/main.py):** Contains FastAPI app initialization, Pydantic schemas, SQLAlchemy DB session dependency, and 15+ REST endpoint routers.
- **Django Core Application (backend/core):** models.py defines relational ORM models; serializers.py handles JSON transformation; views.py exposes REST Framework ViewSets.
- **Automated Audit Suite:** Includes exhaustive_live_audit.py, full_dom_live_audit.py, and playwright_live_audit.py for end-to-end automated UI & API regression testing.

6. GitHub Progress and Commit History

The project development repository trajectory is recorded across five key git commit milestones from initial setup to final production hardening:

Commit Hash & Date	Author	Commit Message & Scope	Key Impact & Changes
07051da Aug 8, 2026	avrojitsaha763-byte	Initial commit	Repository initialization with project README.md overview.
f7f5e25 Aug 9, 2026 (03:58)	avrojitsaha763-byte	Initial commit - NeoRent Rental Platform (Audited & Hardened)	Massive foundational check-in: 75 files changed (+14,698 lines). Established Django backend, FastAPI microservice, SQLite database, React components, and Playwright live audit scripts.
95be379 Aug 9, 2026 (06:25)	avrojitsaha763-byte	feat: complete UI/UX overhaul, 10 WOW features, landing page, and video hero background	Major feature expansion: 24 files changed (+3,362 lines). Implemented 10 WOW features including Video Hero background, Voice Command Panel, NeoBot AI, Risk Oracle, Timeline, Canvas Confetti, and Countdown Timers.
7897a74 Aug 9, 2026 (06:47)	avrojitsaha763-byte	feat: add Admin sign-up tab with neorent435@gmail.com verification and error handling	Security & Auth enhancement in AuthFlow.jsx (+85 lines). Added strict email domain authorization check for administrator registration.
0c9138a Aug 9, 2026 (06:56)	avrojitsaha763-byte	fix: update Admin sign-up email placeholder to generic format	UX polish in AuthFlow.jsx. Updated placeholder text to prevent exposing specific admin email addresses while retaining backend validation.

7. Challenges Faced, Bugs, Issues & Solutions

During the development of NeoRent, several complex technical hurdles, concurrency bottlenecks, and integration bugs were encountered and systematically fixed. Below is a detailed post-mortem of the top 5 challenges:

Challenge 1: CORS & Asynchronous Sync Failures between Vite & FastAPI

Symptom / Bug: During initial frontend integration, API requests executed from the Vite dev server (port 5173/3000) to the FastAPI backend (port 8000) failed with Cross-Origin Resource Sharing (CORS) preflight errors and unexpected network drops.

Root Cause: FastAPI did not have explicitly permitted headers and HTTP origins configured for Vite's dynamic dev server ports.

Resolution & Fix: Implemented FastAPI's CORSMiddleware in backend/main.py with explicit configuration: `allow_origins=['*'], allow_credentials=True, allow_methods=['*'], allow_headers=['*']`. Additionally, implemented optimistic state fallback in AppContext.jsx so that if the backend API is unreachable, local storage instantly fulfills state mutations without crashing the UI.

Challenge 2: Unauthorized Admin Sign-Up Security Vulnerability

Symptom / Bug: The original authentication component allowed any user to toggle role to 'ADMIN' during sign-up, potentially granting unauthorized access to IoT smart locks and financial invoices.

Root Cause: Lack of server-side and client-side verification on administrator registration forms.

Resolution & Fix: Updated AuthFlow.jsx in commit 7897a74 to enforce strict domain verification. Admin registration requests check the email against allowed administrator emails (neorent435@gmail.com). Invalid attempts display an explicit error alert. Placeholder text was sanitized in commit 0c9138a.

Challenge 3: Dynamic Price Calculation across Variable Periodicities & Multi-Currency

Symptom / Bug: Rental orders with custom attribute variants (e.g. 4K Camera + Extra Lens) calculating across non-daily periodicities (Hourly, Weekly, Nightly) produced incorrect total pricing and miscalculated security deposits.

Root Cause: Heterogeneous price structures—some items billed hourly, others daily or weekly—mixed with flat security deposits caused mathematical scaling errors when changing durations.

Resolution & Fix: Built a unified calculation engine in ProductConfigModal.jsx and main.py. Periodicity rate multipliers (Hourly = hrs/24, Weekly = weeks, Nightly = nights) were standardized, and security deposit was isolated as a flat non-discountable line item. Integrated currency conversion rates dynamically.

Challenge 4: SQLite Database Thread Concurrency Locks under Async Workloads

Symptom / Bug: High-frequency async REST requests from the React frontend caused SQLite to throw `sqlite3.OperationalError: database is locked`.

Root Cause: SQLite default lock timeout and single-threaded access mode collided with FastAPI's asynchronous multi-threaded execution model.

Resolution & Fix: Configured SQLAlchemy engine with `connect_args={'check_same_thread': False}` and set pragmas for write-ahead logging (WAL mode). Wrapped DB sessions in FastAPI dependency functions (`get_db()`) ensuring strict session closure via `try...finally db.close()`.

Challenge 5: Mobile Autoplay Restrictions for Video Hero Background

Symptom / Bug: On iOS Safari and mobile Chrome devices, the Video Hero background failed to play, rendering a black screen over the landing page hero area.

Root Cause: Mobile browser media policy blocks video autoplay unless the video is explicitly muted and includes inline playback attributes.

Resolution & Fix: Refactored VideoHero.jsx to include `playsInline`, `muted`, `autoPlay`, and `loop` attributes on the HTML5 tag. Added a high-definition poster image fallback (`hero.png`) to ensure instant visual presentation if video decoding is delayed.